Spec-Driven Development (SDD) on xstockstrat

A 1–3 minute walkthrough of how every feature in the xstockstrat platform is built — from a one-line user story to production deploy — using AI agents under explicit human gates.

Video Outline (the spine)

Use this as the narration outline. Each beat ties to a section below.

Time	Beat	What to show
0:00 – 0:15	**The problem.** Multi-service codebases drift. Specs go stale. Agents hallucinate file paths.	Logo, title card "Spec-Driven Development".
0:15 – 0:35	**The pattern.** Five phases: story → review → spec → review → execute. Every phase produces a checked-in artifact. Every transition requires a gate.	Diagram: the five-phase loop with gates between them.
0:35 – 1:00	**Phase 1 — Story.** A human types one sentence. An agent expands it into a product spec with affected services, governance gates, and acceptance criteria. A second agent reviews it.	Screen: `/sdd-story add-rsi-alert "..."` and the generated `product-spec.md`.
1:00 – 1:25	**Phase 2 — Spec.** A planning agent searches the codebase. Every step it writes cites a real file path found by grep. No invented references.	Screen: the implementation spec with file paths and line numbers highlighted.
1:25 – 1:55	**Phase 3 — Execute.** Steps run one at a time. Each step opens its own PR into the feature branch. Each step requires human confirmation before any write.	Screen: `/sdd-execute next`, the discovery → confirmation → PR loop.
1:55 – 2:25	**The proof.** Every artifact is checked in: `feature.md`, `product-spec.md`, `implementation-spec.md`, `context.md`. CI promotes status from `code-completed` to `launched` automatically.	Screen: `docs/roadmap/features/004-make-repo-public-secure/` directory listing.
2:25 – 2:50	**Why this scales.** New agent sessions read `context.md` and continue. No conversation memory required. Status updates happen in CI. Humans review intent, the harness enforces invariants.	Diagram: session N → context.md → session N+1.
2:50 – 3:00	**Outro.** "Every feature in this repo was built this way." Link to `docs/roadmap/features/`.	Title card with GitHub URL.

Section 1 — The Problem That SDD Solves

Three failure modes show up in any AI-assisted codebase past trivial size:

1. **Spec drift.** A user story turns into chat history. The chat history gets summarized. By the time someone implements it, half the requirements are lost.
2. **Hallucinated references.** An agent confidently writes "edit `services/foo/main.go:42`" — except `main.go` is `foo.go` and the function is on line 60.
3. **Status rot.** Feature trackers say "in progress" for the same feature that shipped to production three weeks ago, because updating the tracker was a manual step.

SDD on xstockstrat eliminates all three at the structural level. It does not depend on agent discipline.

Section 2 — The Five Phases

Each phase is a separate skill (slash command) in `.claude/skills/`. Each produces a markdown artifact under `docs/roadmap/features/NNN-<slug>/`.

```
/sdd-story         →  product-spec.md            (status: draft)
/sdd-review        →  product-spec.md approved   (status: spec-ready)
/sdd-spec          →  implementation-spec.md     (status: implementation-ready)
/sdd-review        →  impl spec advisory pass    (status unchanged)
/sdd-execute       →  per-step PRs               (status: in-progress → code-completed)
/promote + CI      →  promotion PR + auto-flip   (status: launched)
```

The lifecycle column is the source of truth. It is the single field a CI workflow uses to know whether a feature is live in production.

Section 3 — Phase 1: Story (`/sdd-story`)

Input: a feature slug and one or two sentences of user intent.

```
/sdd-story add-rsi-alert "Alert me when RSI crosses 70 on any symbol in my watchlist."
```

The agent creates two files:

- **`feature.md`** — the cover sheet: lifecycle status (`draft`), branch name (`feature/add-rsi-alert`), created date, reviewer roles pulled from `docs/runbooks/reviewer-registry.md` .
- **`product-spec.md`** — the requirements: functional requirements numbered FR-1, FR-2, …, governance gates (proto changes? config keys? DB migrations? approval required?), acceptance criteria.

Gate to advance: `/sdd-review <slug> product-spec`. A reviewer agent — a separate skill — reads the product spec and flips status from `draft` to `spec-ready`, or returns it with comments. Status changes are mechanical: the skill rewrites the lifecycle row.

Section 4 — Phase 2: Spec (`/sdd-spec`)

This is where hallucination is structurally prevented.

The planning skill is configured with `agent: general-purpose` and `effort: high`. It runs as a forked sub-agent with `Read`, `find`, and `grep` tools but **no Write**. Its only job is to search the codebase for evidence and produce an `implementation-spec.md` where every numbered step cites:

- An exact file path (`services/xstockstrat-indicators/app/registry.py`)
- An exact symbol name (`def register_formula(...)`)
- An exact line range (`L42–L67`)

If the planner cannot find a referenced symbol via grep, it **must** say so explicitly. The skill prompt is explicit:

> "Every step you write must cite evidence found in the codebase via Read, find, or grep. Never invent a file path, function name, struct name, or line number. If you cannot find something, say so explicitly."

The result: an executor agent (Phase 3) reading the spec can verify each claim before writing a single character of code.

Section 5 — Phase 3: Execute (`/sdd-execute`)

The executor takes one step at a time. Each step is its own PR.

Boot sequence at every session: 1. Resolve the feature directory (find by slug). 2. Read `implementation-spec.md` for steps and current state. 3. Read `context.md` — the append-only session log. This is the agent's memory. 4. Find the next un-done step.

Per-step loop: 1. **Discover** — re-read the cited files, confirm the symbols are still where the spec says they are. 2. **Confirm** — print the planned changes and wait for the user to type "go" before any write. 3. **Write** — make the edits. 4. **Verify** — run the verification commands from the spec (tests, lint, type check). 5. **Commit + PR** — create branch `feature/<slug>/step-N`, push, open a PR targeting `feature/<slug>`. Stop. Print the PR URL.

The user merges the step PR, then runs `/sdd-execute <slug> next` for step N+1.

No step skips confirmation. No step writes silently. No step accumulates uncommitted state across sessions.

Section 6 — Memory Across Sessions: `context.md`

Conversation history is not a reliable substrate for multi-session work. Days pass. The session compacts. A new agent loads up.

Every SDD skill appends to `context.md` on every meaningful action:

```
## 2026-05-11 — Step 7 complete

- Wired `GH_PAT_SCAN` token into the secret-scan job (trufflehog + gitleaks).
- Decision: rejected the "scan only changed files" optimization — full-history scan
  is required per FR-3 in product-spec.md.
- Files modified: .github/workflows/ci.yml (L82–L110), .gitleaks.toml (new).
- Next: Step 8 — update CONTRIBUTING.md with security-audit reference.
```

A new session re-reads `context.md` and picks up exactly where the last one left off. No re-asking, no re-explaining, no drift.

Section 7 — The Status Loop Is Mechanical

Three CI mechanisms keep the lifecycle field honest:

1. **`/promote` skill** detects features at `code-completed` and lists them in the promotion PR description.
2. **`ci-validate-feature-status.yml`** runs on every push to `main`. It parses the merge commit, finds the promoted features, and rewrites their `feature.md` files: status to `launched`, `**Committed to main**` to the SHA, `**Launched date**` to today. Commits the change back to `main`.
3. **Structural tracking fields** (commit SHA + launch date) make production audits a grep: *"What shipped on 2026-05-12?"* → search.

Humans never type `launched` into a feature.md. The harness does it.

Section 8 — What This Looks Like in the Repo

Browse `docs/roadmap/features/004-make-repo-public-secure/` for a complete, launched feature:

- `feature.md` — status history table from `idea` to `launched`, with the promoted commit SHA.
- `product-spec.md` — the requirements that drove the work.
- `implementation-spec.md` — 11 numbered steps with file paths and line ranges.
- `context.md` — every session log entry from 2026-05-10 through 2026-05-12.

The directory **is** the project plan, the implementation record, and the audit trail. There is no parallel "project tracker" to drift from.

Section 9 — Why This Generalizes

Nothing about SDD is specific to stock strategies. The pattern transfers to any sufficiently complex codebase where:

- Multiple services have interlocking contracts.
- Multiple sessions of work span days or weeks.
- Multiple humans review intent, but writing the code itself is high-leverage to delegate.

The reusable pieces in this repo:

- The seven SDD skills in `.claude/skills/` — drop-in, parameterized by your repo's feature directory layout.
- The lifecycle states (`idea` → `draft` → `spec-ready` → `implementation-ready` → `in-progress` → `code-completed` → `launched`).
- The "every step cites grep evidence" rule as a hard-coded prompt constraint.
- CLAUDE.md files at every directory level as the agent-readable project description.

The non-reusable pieces — what's specific to xstockstrat — are the reviewer registry, the proto-versioning runbook, and the service registry. Everything else is template.

Outro

Every feature in this repo was built this way. Open `docs/roadmap/features/` and read any `context.md` to see the actual session-by-session history of agents writing code under human gates.

Repo: `github.com/davcs86/xstockstrat` **Pattern docs:** `docs/runbooks/feature-workflow.md`
Reusable skills: `.claude/skills/`